

■ .NET 8 AI Migration Assistant Report

Project:	demo_project
Path:	C:\Users\grith\Desktop\Infinite\demo_project
Generated at:	2026-06-09 10:32:59
Overall Risk Rating:	High (71/100)
Migration Readiness:	29%
Estimated Effort:	2-4 Weeks (High Effort)

1. Executive Summary

This migration report evaluates the structure, dependencies, and code configuration of the application located at C:\Users\grith\Desktop\Infinite\demo_project. The assistant scanned legacy codebases, computed risk scores, and generated remediation strategies using Gemini models.

2. Migration Roadmap

- **Phase 1: Project Preparation** - Create full system backups and code checkpoints.
- **Phase 2: Target Upgrades** - Update build dependencies and change TARGET framework to net8.0.
- **Phase 3: Package Refactoring** - Upgrade legacy assemblies and NuGet dependencies.
- **Phase 4: API Remappings** - Code refactoring of WebForms/WCF bindings to Minimal APIs and gRPC.

3. Risk Blocker Summary

• **[High] WCF**

Remediation: WCF is not natively supported in .NET 8. Migrate services to gRPC, CoreWCF, or ASP.NET Core Minimal APIs.

• **[Critical] WebForms**

Remediation: ASP.NET WebForms is not supported in .NET 8. Rewrite interface components using ASP.NET Core Razor Pages, Blazor, or a modern SPA framework (React/Vue) with Web API.

• **[Medium] System.Web**

Remediation: System.Web is deprecated. Replace HttpContext.Current usage with dependency injection of IHttpContextAccessor. Migrate handlers/modules to ASP.NET Core Middleware.

- ***[Medium] Legacy Packages***

Remediation: Upgrade legacy NuGet packages to modern versions targeting .NET 8. Legacy packages found: EntityFramework, log4net, Microsoft.AspNet.Mvc, Microsoft.AspNet.Razor, Microsoft.AspNet.WebApi, Microsoft.AspNet.WebPages, Microsoft.AspNet.Web.Optimization, AutoFac.Mvc5, AjaxControlToolkit

- ***[Low] Config Complexity***

Remediation: Convert XML configurations (Web.config/App.config) to JSON-based appsettings.json. Migrate WCF endpoint configurations to code-based initialization.

4. File-by-File AI Suggestions

File: Default.aspx.cs

Summary: WebForms code-behind 'Default.aspx.cs' with Page_Load lifecycle, ViewState usage, Session state, HttpContext.Current access, and direct ADO.NET database operations.

Strategy: 1. Convert Page class to a Blazor component @code block. 2. Replace Page_Load with OnInitializedAsync(). 3. Replace ViewState with component-level fields/properties. 4. Replace Session access with IDistributedCache or cascading parameters. 5. Replace HttpContext.Current with injected IHttpContextAccessor. 6. Replace SqlConnection/SqlCommand with EF Core DbContext.

Migrated Equivalent Preview:

```
@page "/"
@using Microsoft.AspNetCore.Http
@using Microsoft.EntityFrameworkCore
@inject AppDbContext Db
@inject IHttpContextAccessor HttpContextAccessor
@inject NavigationManager NavigationManager

<PageTitle>Dashboard</PageTitle>

@code {
    private List<OrderDto> orders = new();
    private string currentUser = "";
    private int totalOrders;
    private decimal totalRevenue;

    protected override async Task OnInitializedAsync()
    {
        // Replaces HttpContext.Current.User
        currentUser = HttpContextAccessor.HttpContext?.User?.Identity?.Name ?? "Anonymous";

        // Replaces ViewState + Page_Load data binding
        orders = await Db.Orders
            .Include(o => o.Customer)
            .OrderByDescending(o => o.OrderDate)
            .Take(100)
            .Select
            //... [code truncated for length in PDF] ...
    }
}
```

File: Global.asax.cs

Summary: ASP.NET Global Application file 'Global.asax.cs' implementing HttpApplication lifecycle events (Application_Start, Application_BeginRequest, Application_Error, Session_Start).

Strategy: 1. Move Application_Start logic to Program.cs builder configuration. 2. Replace Application_BeginRequest/EndRequest with ASP.NET Core middleware. 3. Replace Application_Error with app.UseExceptionHandler() middleware. 4. Replace Session_Start with session configuration in Program.cs. 5. Replace Application/Session state with DI-registered singleton/scoped services.

Migrated Equivalent Preview:

```
// Program.cs – replaces Global.asax.cs entirely

using LegacyApp.Middleware;

var builder = WebApplication.CreateBuilder(args);
```

```
// Service registration (replaces Application_Start)
builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>(o =>
o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
builder.Services.AddMemoryCache();
builder.Services.AddSession(o => o.IdleTimeout = TimeSpan.FromMinutes(20));
builder.Services.AddSingleton<IAppMetrics, AppMetrics>();

var app = builder.Build();

// Middleware pipeline (replaces Application_BeginRequest/EndRequest)
app.UseMiddleware<RequestTimingMiddleware>();
app.UseExceptionHandler("/Error");
app.UseHsts();
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseA
//... [code truncated for length in PDF] ...
```

File: Global.asax

Summary: Legacy .NET component 'Global.asax' requiring upgrade to .NET 8.

Strategy: 1. Update target framework to net8.0 in .csproj. 2. Replace legacy assembly references with modern NuGet packages. 3. Update namespace imports for .NET 8 compatibility.

Migrated Equivalent Preview:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
<PropertyGroup>
<TargetFramework>net8.0</TargetFramework>
</PropertyGroup>
</Project>
```

File: Default.aspx

Summary: WebForms page 'Default.aspx' using server controls (GridView, UpdatePanel, ScriptManager), ViewState for state management, and ASPX markup with code-behind pattern.

Strategy: 1. Replace .aspx markup with a Blazor component (.razor) or Razor Page. 2. Replace with a Blazor table component or MudBlazor DataGrid. 3. Replace with Blazor's built-in re-rendering. 4. Remove ViewState — use Blazor component state (@code { }) instead. 5. Replace code-behind event handlers (Page_Load, Button_Click) with Blazor lifecycle methods.

Migrated Equivalent Preview:

```
@page "/"
@using LegacyApp.Data
@inject IOrderService OrderService
@inject IHttpContextAccessor HttpContextAccessor

<PageTitle>Enterprise Dashboard</PageTitle>

<div class="container">
<h1>Enterprise Dashboard</h1>

<MudTable Items="@orders" Hover="true" Striped="true" Loading="@isLoading"
```

```
Pagination="new MudTablePager { PageSizeOptions = new[] { 10, 25, 50 } }">
<HeaderContent>
<MudTh>Order #</MudTh>
<MudTh>Customer</MudTh>
<MudTh>Date</MudTh>
<MudTh>Total</MudTh>
<MudTh>Status</MudTh>
<MudTh>Actions</MudTh>
</HeaderContent>
<RowTemplate>
<MudTd>@context.OrderId</MudTd>
<MudTd>@context.CustomerName</MudTd>
<MudTd>@context.OrderDate.ToSt
//... [code truncated for length in PDF] ...
```

File: Web.config

Summary: XML configuration file 'Web.config' containing connection strings, system.web settings, system.serviceModel WCF bindings, Entity Framework configuration, and custom HTTP modules.

Strategy: 1. Move to appsettings.json under "ConnectionStrings" key. 2. Move to appsettings.json. 3. Replace authentication/authorization with ASP.NET Core Identity middleware. 4. Replace WCF config with code-based endpoint registration. 5. Replace config with EF Core registration in Program.cs. 6. Replace with ASP.NET Core middleware in Program.cs.

Migrated Equivalent Preview:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=.\SQLEXPRESS;Database=LegacyAppDB;Trusted_Connection=true;TrustServerCertificate=true",
    "ReportingDB":
    "Server=.\SQLEXPRESS;Database=ReportingDB;Trusted_Connection=true;TrustServerCertificate=true"
  },
  "AppSettings": {
    "MaxPageSize": 100,
    "EnableAuditLog": true,
    "SmtpServer": "smtp.company.internal"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}

// Program.cs – replaces Web.config system sections:
// var builder = WebApplication.CreateBuilder(args);
//
// // Replaces <system.web><authentication mode="Forms">
// builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
// .AddCookie(options => { options.L
//... [code truncated for length in PDF] ...
```

File: Controllers\HomeController.cs

Summary: Legacy ASP.NET MVC controller or HTTP module 'HomeController.cs' using HttpContext.Current, Session, Application state, Cache API, and ConfigurationManager.

Strategy: 1. Replace HttpContext.Current with constructor-injected IHttpContextAccessor. 2. Replace System.Web.Mvc.Controller with Microsoft.AspNetCore.Mvc.Controller. 3. Replace ConfigurationManager with IConfiguration. 4. Replace ASP.NET Cache with IMemoryCache or IDistributedCache. 5. Replace Session direct access with HttpContext.Session extension methods. 6. Replace Application state with a singleton service registered in DI.

Migrated Equivalent Preview:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Caching.Memory;
using Microsoft.Extensions.Configuration;
using Microsoft.EntityFrameworkCore;

namespace LegacyApp.Controllers;

/// <summary>Home controller migrated to ASP.NET Core with dependency injection.</summary>
public class HomeController : Controller
{
    private readonly IHttpContextAccessor _httpContextAccessor;
    private readonly IMemoryCache _cache;
    private readonly IConfiguration _configuration;
    private readonly ApplicationDbContext _db;

    public HomeController(
        IHttpContextAccessor httpContextAccessor,
        IMemoryCache cache,
        IConfiguration configuration,
        ApplicationDbContext db)
    {
        _httpContextAccessor = httpContextAccessor;
        _cache = cac
        //... [code truncated for length in PDF] ...
    }
}
```

File: Services\OrderService.svc

Summary: WCF service host file 'OrderService.svc' that routes SOAP requests to the service implementation. This file defines the ServiceHost directive used by IIS to activate the WCF service.

Strategy: 1. Remove the .svc file entirely — .NET 8 does not use .svc hosting. 2. Register the service as a Minimal API endpoint in Program.cs. 3. If SOAP compatibility is required, use CoreWCF NuGet package. 4. For new clients, prefer gRPC with Protobuf contracts.

Migrated Equivalent Preview:

```
// .svc file is eliminated in .NET 8.
// Service is registered in Program.cs as a Minimal API:

// Program.cs
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();
app.MapGroup("/api/orderservices").MapOrderServiceEndpoints();
app.Run();
```

File: Services\OrderService.svc.cs

Summary: WCF service implementation 'OrderService.svc.cs' with [ServiceContract] and [OperationContract] attributes, using ADO.NET for data access via SqlConnection and ConfigurationManager.

Strategy: 1. Replace [ServiceContract] interface with ASP.NET Core Minimal API endpoint group. 2. Replace [OperationContract] methods with MapGet/MapPost/MapPut/MapDelete handlers. 3. Replace [DataContract]/[DataMember] DTOs with C# records. 4. Replace ConfigurationManager with IConfiguration via dependency injection. 5. Replace raw SqlConnection/SqlCommand with Entity Framework Core DbContext. 6. Add proper input validation with FluentValidation or Data Annotations.

Migrated Equivalent Preview:

```
using Microsoft.AspNetCore.Http.HttpResults;
using Microsoft.EntityFrameworkCore;

namespace LegacyApp.Endpoints;

/// <summary>Order data transfer object.</summary>
public record OrderDto(int OrderId, string CustomerName, DateTime OrderDate, decimal
TotalAmount, string Status);

/// <summary>Request payload for creating a new order.</summary>
public record CreateOrderRequest(int CustomerId, List<OrderItemRequest> Items);
public record OrderItemRequest(int ProductId, int Quantity, decimal UnitPrice);

/// <summary>Standard API response envelope.</summary>
public record ServiceResponse(bool Success, string Message, int? OrderId = null);

/// <summary>Minimal API endpoints replacing the WCF OrderService.</summary>
public static class OrderEndpoints
{
    public static RouteGroupBuilder MapOr
    //... [code truncated for length in PDF] ...
}
```